

Advanced Game Engine (UE5) – Final Project Report (Template)

Project Title: {{Platformers}}

Team Name: {{Team_9}}

Semester / Year: {{Spring semester_2025}}

Instructor: {{이중호, Rafiq}}

UE Version: {{5.0.3}}

Team Members & Responsibilities

Name	Student ID	Role	Responsibilities / Deliverables
김영호(KimYoungHo)	5646372	Level B Design	Designed and implemented Level B
황주현(HwangJuHyeon)	5646281	Level A Design	Designed and implemented Level A
서현규(SeoHyunGyu)	5646125	Level C Design	Designed and implemented Level C
이창민(LeeChangMin)	5702600	Level D Design	Designed and implemented Level D

Abstract

This project is a UE5-based 3D platformer developed by a team of four, in which each member independently designed a platformer level under a shared minimal constraint: every level must include a clearly defined start point and end point.

Rather than organizing levels by difficulty or progression, the project focuses on freely experimenting with different platformer gimmicks and interactions. The four levels are labeled sequentially as Level A, B, C, and D, without implying any hierarchy in difficulty or complexity.

All levels are built upon a shared character framework and core control scheme, ensuring a consistent player experience across the project. However, to support level-specific

gameplay requirements, certain levels extend the base character logic with additional input handling, demonstrating controlled variation within a common system.

Through this modular level structure, the project highlights an understanding of Unreal Engine's gameplay architecture, blueprint-based system design, and collaborative development practices, while allowing individual creativity within a unified technical framework.

Project Concept & Genre

This project adopts a 3D platformer format as a framework for exploring diverse gameplay mechanics within Unreal Engine 5.

Rather than following a predefined difficulty curve or linear progression, the design intentionally limits formal constraints to a simple requirement: each level must define a clear start point and end point. This decision was made to encourage independent exploration of platformer mechanics while maintaining a minimal level of structural consistency across the project.

Each level was developed as a self-contained space, allowing team members to focus on implementing and testing specific gimmicks without being restricted by overarching balance or progression concerns. As a result, the project functions less as a traditional level-based game and more as a collection of focused platformer experiments unified by shared technical foundations.

From a genre standpoint, the project remains a 3D platformer, but its primary goal lies in technical understanding and system implementation rather than commercial polish. This conceptual direction supports learning objectives related to gameplay architecture, blueprint-driven development, and collaborative production workflows in Unreal Engine.

Core Gameplay Loop

The core gameplay loop of the project centers on navigating a self-contained platformer level from a defined start point to a designated end point.

Upon entering a level, the player gains control of a shared third-person character and immediately begins interacting with level-specific platforming elements. These elements include moving platforms, traps, randomized paths, and other gimmicks that challenge the player's timing, positioning, or decision-making.

Failure typically results in the player being repositioned to a checkpoint or the start of the level, encouraging repeated attempts and gradual familiarization with the active gimmick. Success is achieved by reaching the end point, which triggers a transition to the next level.

Because each level is designed as an independent experiment rather than a step in a difficulty progression, the gameplay loop remains consistent across all levels while the mechanics encountered within that loop vary. This structure allows players to quickly understand the objective while focusing their attention on adapting to new interactions introduced by each level.

Unreal Engine Architecture (Diagram + Explanation)

The project is built upon Unreal Engine 5's standard gameplay framework, primarily utilizing the default third-person `GameMode`, `PlayerController`, and `Character` classes as a shared foundation across all levels. This approach ensures consistency in core player controls and movement behavior throughout the project.

Most gameplay variation is introduced at the level level, where individual maps define their own gimmicks, obstacles, and spatial layouts. By concentrating gameplay differences within the level design itself, the project maintains a stable and predictable core system while allowing each level to express unique mechanics.

While most levels rely on the shared control scheme, certain levels require additional player input to support specific gameplay interactions. Rather than modifying the global character or controller logic, these requirements are handled by introducing a separate `GameMode` and `PlayerController` that are assigned only to the relevant levels.

This selective separation allows level-specific input logic to be managed independently, preventing unintended side effects on other levels. Overall, the architecture balances shared systems and controlled variation, enabling modular level development, parallel team workflows, and maintainable system expansion.

Blueprint/C++ Systems Design

The project's gameplay systems are implemented primarily using Unreal Engine's Blueprint system, without relying on custom C++ code. This design choice emphasizes rapid iteration, visual clarity, and accessibility in a team-based development environment, while fully utilizing Unreal Engine's gameplay framework.

At a structural level, all levels share a common character foundation and basic input scheme, including movement and jumping. Core gameplay behavior is kept consistent across the project, while experimental mechanics are encapsulated within level-specific Blueprint Actors. This separation ensures that individual gimmicks remain isolated and do not introduce unintended side effects across levels.

One of the most widely used system patterns in the project is trigger-based interaction. Collision triggers are used to detect player presence and initiate state changes such as platform activation, floor collapse, information display, or level progression. This

interaction model is applied across multiple levels, including Level A (Hwang Juhyeon), Level B (Kim Youngho), and Level C (Seo Hyungyu), with each level adapting the same core concept to different gameplay purposes.

Randomized and conditional logic is used selectively to influence player decision-making rather than altering physical level structures. Blueprint nodes such as Random Integer, Branch, and Switch are employed to control conditional outcomes based on player interaction. In Level A (Hwang Juhyeon), randomization governs probability-based teleportation and path selection mechanics. In contrast, Level B (Kim Youngho) applies randomization at the information level rather than the environment itself. When the player enters a trigger area, one of several predefined questions is selected at random and presented. The player must interpret the information and choose the correct door to proceed, while the physical layout of platforms remains fixed.

Time-pressure systems are incorporated to shape player behavior under constrained conditions. Level B introduces a continuously advancing death wall that forces players to process information quickly and commit to decisions under pressure. Level C similarly employs delayed trap activation, where success depends on timing and anticipation rather than pure execution. These systems shift the platforming experience toward situational judgment and risk management.

While most levels operate under a shared control scheme, certain gameplay requirements necessitate level-specific input handling. Level D (Lee Changmin) introduces additional player input to manage stage progression and skipping functionality. Instead of modifying the global input system, a separate GameMode and PlayerController are assigned exclusively to this level. This approach preserves the stability of shared systems while enabling controlled extensions where required.

Overall, the Blueprint/C++ system design balances consistency and flexibility. By maintaining shared core systems and isolating experimental mechanics at the level level, the project supports modular development, clear division of responsibility among team members, and maintainable system expansion without compromising system stability.

Animation System

The project utilizes Unreal Engine's default third-person animation system as the foundation for character movement and interaction. Standard animation assets such as idle, walking, running, and jumping are used to ensure stable and predictable player feedback across all levels.

No custom animation blueprints or complex blending logic were introduced, as the primary focus of the project lies in gameplay mechanics and system experimentation rather than animation-driven expression. By relying on Unreal Engine's built-in animation framework,

the project maintains consistency in character behavior while allowing development effort to be concentrated on level-specific gimmicks and interactions.

Physics / Chaos Features

Physics interactions in the project are handled using Unreal Engine's standard collision and physics systems. Basic collision detection is employed for player movement, platform interaction, traps, and failure conditions.

Advanced Chaos physics features such as rigid body simulation, destruction systems, or complex physics constraints were intentionally not used. Given the scope and goals of the project, physics behavior is kept deterministic and lightweight to ensure consistent gameplay outcomes and to avoid introducing unnecessary system complexity.

Rendering, Lighting, VFX (Niagara, Lumen/Nanite if used)

The project uses Unreal Engine's default rendering pipeline with standard real-time lighting to support clear visual readability of level geometry and gameplay elements. Lighting is kept simple and functional, prioritizing player visibility and obstacle recognition over cinematic presentation.

Advanced rendering features such as Nanite virtualized geometry, Lumen global illumination, or Niagara-based visual effects were not a primary focus of this project. Instead, the visual presentation is designed to remain lightweight and consistent, ensuring stable performance and reducing visual noise that could interfere with gameplay clarity.

Level & World Design (Maps, Navigation, Streaming/World Partition if used)

The project adopts a modular level design approach, where each level functions as an independent gameplay space rather than forming a continuous world or difficulty-based progression. All levels are connected sequentially, but each maintains clear boundaries in layout, mechanics, and design intent. This structure supports parallel development among team members and allows each level to act as a focused experiment in platformer gameplay design.

All levels share a simple and consistent spatial objective: guiding the player from a clearly defined start point to an end point. Within this shared constraint, each level explores different gameplay ideas through variations in layout structure, interaction design, and environmental pressure. The naming convention of Level A, B, C, and D is used purely for identification purposes and does not imply difficulty order or progression hierarchy.

Level A (Hwang Juhyeon) begins on small floating platforms suspended in the air, immediately creating tension through the risk of falling. If the player falls, they are instantly

respawned at the starting point, reinforcing the importance of precision and spatial awareness. The level introduces randomized path selection followed by a probability-based vertical teleportation section, requiring players to identify the correct route in order to advance to the next stage.

Level B (Kim Youngho) is designed to test the player's memory and decision-making under continuous time pressure. As a threatening force approaches from behind, the player must recall previously presented information and solve problems within a limited time frame to identify the correct door and proceed. Rather than emphasizing mechanical difficulty, the level focuses on cognitive judgment, information recall, and rapid decision-making under stress.

Level C (Seo Hyungyu) emphasizes forward momentum and timing-based traversal. The player is required to continuously move across horizontally moving platforms while navigating falling floor traps that trigger during progression. Vertical elevators are used to structure spatial flow and guide the player steadily toward the destination in a clear and linear manner.

Level D (Lee Changmin) deliberately subverts traditional platformer expectations by presenting stages that are intentionally impossible to clear. Instead of reaching a goal through conventional means, the player progresses by skipping stages using a dedicated input key. The level consists of five independent stages contained within a single map, each occupying an isolated space to prevent interference. PlayerStart actors are placed in each stage to manage transitions and progression.

UI/UX & Audio

The project adopts a minimal UI/UX approach to ensure that player attention remains focused on gameplay mechanics and level-specific challenges. On-screen elements are intentionally limited, with player guidance delivered primarily through spatial layout, environmental cues, and interactive objects rather than explicit interface elements.

No dedicated audio system or sound effects were implemented in this project. This decision was made deliberately, as the core gameplay relies on visual information, timing, and spatial judgment rather than auditory feedback. By excluding audio elements, the design emphasizes player observation and interpretation of visual cues, aligning with the project's focus on platformer mechanics and cognitive challenge.

Performance Optimization (Profiling Evidence + Changes + Results)

Given the limited scope and controlled complexity of the project, performance optimization was addressed through preventive design choices rather than extensive post-development

tuning. Levels are composed of lightweight geometry and a limited number of actors, reducing runtime overhead.

Gameplay logic is primarily event-driven, relying on trigger-based activation instead of continuous per-frame calculations. This minimizes unnecessary performance costs and ensures stable execution across all levels. No major performance bottlenecks were observed during development, and therefore detailed profiling or targeted optimization was not required.

Overall performance stability was achieved by avoiding expensive rendering features, maintaining simple collision logic, and isolating gameplay systems within individual levels. This approach resulted in consistent frame rates throughout gameplay without the need for additional optimization passes.

Challenges & Solutions

Challenge 1: Integrating Independently Designed Levels and Blueprints

One of the primary challenges faced by the team was integrating multiple independently designed levels and Blueprint systems into a single Unreal Engine project. Since each team member developed their level in isolation, differences in map structure and Blueprint usage had to be carefully managed during integration.

This issue was addressed by importing each level and its associated Blueprints incrementally, preserving their original structure. By applying and testing each component step by step, the team was able to identify conflicts early and ensure stable integration without disrupting existing gameplay systems.

Challenge 2: Resolving Shared Asset and Character Blueprint Conflicts

Another significant challenge involved conflicts arising from shared assets, particularly the use of a common Character Blueprint across multiple levels. As different levels introduced their own gameplay logic, unintended interactions and logic collisions occasionally occurred, leading to unexpected behavior.

To resolve this, the team analyzed Blueprint dependencies and separated level-specific logic from shared character functionality. By clearly defining the responsibilities of common systems and isolating experimental mechanics within individual levels, the overall project stability was significantly improved.

Challenge 3: Managing Stage Transitions in a Multi-Stage Level (Case Study: Lee Changmin)

A unique technical challenge was encountered during the development of Level D, primarily handled by team member Lee Changmin. Due to the stage-based structure of the level, implementing seamless transitions between multiple stages within a single map proved difficult.

Initially, stage transitions were managed by storing stage start positions as vector coordinates. However, this approach required manual updates whenever a new stage was added and was difficult to manage as the number of stages increased. To address this limitation, the system was redesigned to utilize multiple PlayerStart actors within the same level.

By using the Get All Actors Of Class node to retrieve all PlayerStart actors and distinguishing them through unique tags, the player could be spawned at the correct stage location dynamically. This solution greatly improved scalability and maintainability while aligning with Unreal Engine's intended workflow.

Results & Final Build State

As a result of the project, each team member was able to design and implement an individual platformer level that experimented with two or more gameplay mechanics. These mechanics include randomized decision-making, trigger-based interactions, timing-based traversal, and stage-based progression.

Despite being developed independently, all levels were successfully integrated into a single Unreal Engine project using a shared technical framework. The final build demonstrates stable gameplay flow across all levels, confirming that the modular design approach effectively supported parallel development and system consistency.

Limitations & Future Work

One limitation of the project is that the overall experience does not fully convey the feeling of a complete game. While individual mechanics were explored in depth, limited development time and scope prevented the addition of narrative context, progression systems, or audiovisual elements that could enhance immersion.

As a result, the project remains closer to a collection of gameplay experiments rather than a fully polished game experience.

For future development, the existing modular structure could be extended to support a more cohesive game experience. Additional levels could be introduced with a gradual difficulty curve, allowing mechanics to build upon one another.

Furthermore, lightweight UI, audio feedback, and progression elements could be integrated to improve player immersion while preserving the current system architecture. These enhancements would transform the project from a set of isolated gameplay experiments into a more unified platformer experience.

References / Assets Used (include licenses where applicable)

Several existing games were referenced for high-level gameplay ideas and player experience.

These references were used strictly as conceptual inspiration, with no direct reuse of assets or code.

- Alt-F4 – Referenced for its puzzle-based progression, where players must identify correct solutions through repeated attempts and failure

- TalesRunner – Referenced for its use of randomly generated platforms and dynamic obstacle layouts

- "Our Cat Has Disabilities" (Korean indie Flash game) – Referenced for its unconventional level design approach and challenge presentation

All assets used in this project were provided by the Unreal Engine default content.

No third-party assets, plugins, or external libraries were used.

Appendix – Build & Run Instructions

Controls: {{ The player uses standard keyboard controls for movement and interaction. Basic movement is handled using **W**, **A**, **S**, **D** keys. Additional context-specific actions are performed using keys such as **E** and **R**, which are explained through in-game prompts when required.}}

How to run the Windows build: {{ To run the project, locate the Windows build folder included in the submission files.

Execute **finalexam.exe** to start the game. No additional installation or configuration steps are required.}}

Known issues: {{ No known issues at the time of submission.}}